ACADEMIC VIVA PREPARATION

FamilyCare System

Final Year University Project Defense & Comprehensive Viva Master Guide

Project Scope: Full Stack Health & Elderly Care
Tech Stack: React 18, Node.js, Express, MySQL

---

1. PROJECT OVERVIEW & MOTIVATION

Q1 What is this project about and what core problem does it solve?

💡 **SIMPLE EXPLANATION (UNDERSTAND THE CONCEPT)**

Working children cannot be at home 24/7 with their aging parents. There is no easy way to track their daily blood pressure/sugar, manage doctor visits, or hire certified caregivers with structured monthly subscriptions. FamilyCare connects Children, Parents, Caregivers, and an Admin into one unified, secure platform.

🎙️ **VIVA-READY ANSWER (WHAT TO SAY TO THE PROFESSOR)**

"Respected panel, FamilyCare is a full-stack, role-based elderly care management and health monitoring platform. It bridges the care-coordination and visibility gap between working adult children, elderly parents, and certified professional caregivers. The system provides daily health vitals logging with automated clinical threshold alerts, caregiver patient capacity management, and secure 30-day care package subscriptions via the PayHere payment gateway."

📂 `Code: frontend/src/App.jsx & backend/src/index.js`

Q2 What are the 4 user roles in your system and what can each role do?

💡 **SIMPLE EXPLANATION**

Admin: Approves caregivers, views platform stats, manages users.
Child: Registers parents, chooses verified caregivers, pays monthly subscriptions via PayHere, and monitors health vitals.
Caregiver: Sets their monthly care package price/features, logs daily health vitals for assigned residents, and manages appointments.
Parent: Simplified view of their assigned caregiver, daily medication schedule, and doctor appointments.

🎙️ **VIVA-READY ANSWER**

*"FamilyCare enforces strict Role-Based Access Control (RBAC) across 4 user tiers: 1. **Admin:** Reviews caregiver credentials, approves accounts, and monitors platform metrics. 2. **Child (Family Member):** Registers elderly parents, browses caregivers, pays monthly care subscriptions, and tracks daily health charts. 3. **Caregiver:** Configures monthly service packages, logs vital signs, and manages appointments within strict capacity limits. 4. **Parent:** Provides a high-accessibility dashboard showing daily schedules, medications, and caregiver contact."*

📂 `Code: backend/src/middleware/auth.js & userRoutes.js`

2. ARCHITECTURE & TECH STACK DEFENSE

Q3 Explain your system architecture. Why decoupled client-server?

💡 SIMPLE EXPLANATION

The frontend (React) and backend (Node.js/Express) are completely separate. The frontend runs in the user's browser as a Single Page App. When it needs data, it sends an HTTP request to the backend REST API. The backend fetches data from MySQL and returns JSON. This separation means we can easily build a mobile app in the future without changing the backend.

🎙️ VIVA-READY ANSWER

"Our system follows a Decoupled 3-Tier Client-Server Architecture: 1. ***Presentation Layer:*** *React 18 SPA with Vite for high-performance reactive UI rendering. 2.* ***Application Layer:*** *Stateless Node.js and Express RESTful API handling authentication, business logic, and PayHere checksum hashing. 3.* ***Data Layer:*** *MySQL relational database ensuring ACID transactional integrity. This decoupled pattern guarantees separation of concerns, faster client-side routing, and future API reusability for native mobile applications."*

📂 `Code: backend/src/index.js & frontend/src/main.jsx`

Q4 Why did you choose MySQL over MongoDB (NoSQL)?

💡 **SIMPLE EXPLANATION**

Healthcare logs and financial transactions need strict rules. A subscription must strictly link to a parent and a caregiver. MySQL provides ACID guarantees (safe transactions) and Foreign Keys with CASCADE/SET NULL rules to prevent corrupted or orphaned records.

🎙️ **VIVA-READY ANSWER**

"Elderly health data and subscription billing require strict ACID compliance (Atomicity, Consistency, Isolation, Durability) and relational integrity. In FamilyCare, entities have strict dependencies (Users → Parents → Subscriptions → Payments). MySQL's foreign key constraints prevent orphaned records and ensure transactional consistency, which is vital for clinical logs and financial audit trails."

📂 `Code: backend/src/config/db.js`

3. AUTHENTICATION, SECURITY & DATABASE

Q5 How does Authentication and JWT Authorization work in your system?

💡 **SIMPLE EXPLANATION**

When the user logs in, the backend checks the password using bcrypt. If valid, the backend signs a JSON Web Token (JWT) containing { userId, role }. The frontend stores this token and includes it in the HTTP header ('Authorization: Bearer ...') for every request. Backend middleware checks this signature before letting the user view private data.

🎙️ **VIVA-READY ANSWER**

"Authentication uses Stateless JSON Web Tokens (JWT) combined with bcryptjs cryptographic password hashing. Upon successful login, the server signs a JWT payload with the user's ID and role. The client includes this token in the 'Authorization: Bearer' header. Backend middleware (verifyToken, requireRole) intercepts each request, verifies the cryptographic signature, and prevents unauthorized cross-role access (e.g. returning 403 Forbidden if a Child tries accessing Admin endpoints)."

📂 `Code: backend/src/controllers/authController.js`

Q6 How do you prevent SQL Injection attacks?

💡 SIMPLE EXPLANATION

We never glue user text directly into SQL strings. We always use question mark placeholders '?' (Parameterized Queries). The database engine pre-compiles the query and treats user text strictly as raw data, never as executable SQL code.

🎙️ VIVA-READY ANSWER

*"We prevent SQL Injection by strictly using Parameterized Prepared Statements via mysql2 (e.g., pool.query('SELECT * FROM users WHERE email = ?', [email])). The SQL statement structure is pre-compiled by the database engine, ensuring user input is treated strictly as literal data rather than executable SQL commands."*

📂 Code: backend/src/controllers/userController.js

4. PAYMENT GATEWAY & CORE BUSINESS LOGIC

Q7 Explain the PayHere Sandbox Payment & Subscription workflow in detail.

💡 **SIMPLE EXPLANATION**

1. Child chooses a caregiver for a parent and clicks "Pay with PayHere".
2. Backend generates a unique Order ID and a secure MD5 Checksum Hash using the merchant secret.
3. PayHere Sandbox popup opens with the test card modal.
4. Upon payment completion, backend verifies the payment, marks the subscription as Active for 30 days, assigns the caregiver to the parent, and saves a receipt.

🎙️ **VIVA-READY ANSWER**

*"The payment lifecycle follows a secure 4-step workflow: 1. **Initiation (/api/users/subscriptions/initiate):** Backend creates a pending subscription record and generates an MD5 checksum hash:*

```
Hash = MD5(merchant_id + order_id + amount + currency + MD5(merchant_secret))
```

*2. **Client Execution:** PayHere JavaScript SDK launches the secure checkout popup with signed parameters. 3. **Verification (/api/users/subscriptions/verify):** Backend verifies the payment signature, activates the subscription for +30 days, assigns the caregiver to the parent, and writes a transaction record into subscription_payments. 4. **IPN Webhook Listener (/payhere-notify):** Listens for asynchronous server-to-server payment notifications directly from PayHere."*

📁 Code: backend/src/config/payhere.js & userController.js

Q8 What is Caregiver Capacity Management and how is it enforced?

💡 SIMPLE EXPLANATION

One caregiver cannot safely care for too many elderly residents at once. We set a maximum capacity limit (e.g. 4 residents). If a caregiver already has 4 patients assigned, the backend rejects new assignments with an error.

🎙️ VIVA-READY ANSWER

"To safeguard elder care quality, the backend enforces a Caregiver Capacity Constraint. Each caregiver profile has an active_residents count and a max_capacity limit (default 4). When an assignment is attempted, the backend checks: active_residents < max_capacity. If the caregiver is at capacity, the API rejects the request with HTTP 400 Bad Request, preventing caregiver burnout and ensuring personalized elder attention."

📂 `Code: backend/src/controllers/caregiverController.js`

5. QUICK REFERENCE MASTER TABLE

Topic	Key Academic Vocabulary to Speak
Architecture	Decoupled 3-Tier Client-Server Architecture with Stateless RESTful APIs
Frontend	React 18 Single Page Application (SPA) with Vite Bundler & Virtual DOM
Database	Relational MySQL in 3rd Normal Form (3NF) with Connection Pooling & ACID compliance
Security	Stateless JWT Tokens, bcryptjs Salting/Hashing, Parameterized Prepared Queries
Payment Gateway	PayHere Sandbox Gateway with MD5 Checksum signature verification and IPN Webhooks

Topic	Key Academic Vocabulary to Speak
Business Logic	Caregiver Capacity Constraints, Daily Vital Clinical Threshold Alerts, 30-Day Subscription Lifecycle

💡 Golden Rule for Tomorrow

Speak with confidence, maintain good eye contact with the professors, explain the *why* behind your design choices, and reference your code files. Best of luck! 🎓